

- Dispensa — Progetto 3 Principiante
 - Gioco "Indovina il Numero"
 - Indice
 - 1. Il modulo random: randint()
 - random.randint(a, b)
 - import — importare un modulo
 - Nel progetto
 - 2. Funzioni con return di tuple
 - Creare una tupla
 - Restituire una tupla da una funzione
 - Unpacking — spaccettare la tupla
 - Accesso per indice
 - 3. Il ciclo while con contatore
 - while condizione: — ripete finché la condizione è vera
 - Nel progetto: limitare i tentativi
 - while True con break
 - Differenza: while condizione vs while True
 - 4. continue — saltare un'iterazione
 - break vs continue
 - 5. Condizionali: confronto numerico
 - Operatori di confronto
 - Nel progetto: dare suggerimenti
 - Validare un range
 - 6. Conversione di tipo: int() e try/except
 - int() — da stringa a intero
 - try/except — gestire gli errori
 - Nel progetto
 - 7. Liste e .append() per raccogliere risultati
 - Liste — ripasso
 - Nel progetto: raccogliere i risultati delle partite
 - 8. List comprehension per filtrare
 - Filtrare gli elementi di una lista
 - Come si legge
 - 9. sum(), len(), min() — statistiche base
 - sum() — somma di tutti gli elementi
 - len() — numero di elementi
 - Media: sum() / len()

- `min()` — valore minimo
- Formattazione con `:.1f`
- 10. Metodi delle stringhe: `.strip()` e `.lower()`
 - `.strip()` — rimuovere spazi
 - `.lower()` — tutto minuscolo
 - Combinazione per normalizzare l'input
 - Catena di metodi
- 11. Riepilogo e consigli
 - Mappa concetti → dove li usi nel progetto
 - Ordine consigliato di implementazione
 - Errori comuni
 - Riferimenti ufficiali

Dispensa — Progetto 3 Principiante

Gioco "Indovina il Numero"

Obiettivo della dispensa: fornirti gli strumenti necessari per completare il progetto. In alcuni casi faremo riferimenti espliciti al codice che verrà usato direttamente nella soluzione.

Indice

1. Il modulo `random`: `randint()`
2. Funzioni con `return` di tuple
3. Il ciclo `while` con contatore
4. `continue` — saltare un'iterazione
5. Condizionali: confronto numerico
6. Conversione di tipo: `int()` e `try/except`
7. Liste e `.append()` per raccogliere risultati
8. List comprehension per filtrare
9. `sum()`, `len()`, `min()` — statistiche base
10. Metodi delle stringhe: `.strip()` e `.lower()`
11. Riepilogo e consigli

1. Il modulo `random`: `randint()`

Il modulo `random` serve per generare valori casuali. In questo progetto usiamo `randint()`.

`random.randint(a, b)`

Restituisce un intero casuale N tale che $a \leq N \leq b$ (entrambi inclusi):

```
import random

random.randint(1, 20)    # es: 14 (un numero tra 1 e 20)
random.randint(1, 100)   # es: 73 (un numero tra 1 e 100)
random.randint(1, 6)     # es: 4  (simula un dado)
```

`import` — importare un modulo

```
import random    # Importa il modulo intero

random.randint(1, 10)  # Usa la funzione con il prefisso "random."
```

L'istruzione `import` deve stare **all'inizio del file**, prima del resto del codice.

Nel progetto

```
def genera_numero(limite):
    """Genera un numero casuale tra 1 e limite (inclusi)."""
    return random.randint(1, limite)

# Uso:
numero_segreto = genera_numero(20)    # Numero tra 1 e 20
numero_segreto = genera_numero(100)   # Numero tra 1 e 100
```

Riferimento: [random.randint\(\)](#)

2. Funzioni con **return** di tuple

Una **tupla** e' una sequenza di valori racchiusa tra parentesi tonde. Le funzioni possono restituire piu' valori usando una tupla.

Creare una tupla

```
coppia = (20, 8)           # Tupla con due elementi
tripla = (1, "ciao", 3.14) # Tipi diversi
```

Restituire una tupla da una funzione

```
def scegli_difficolta():
    if scelta == "1":
        return (20, 8)      # Restituisce DUE valori insieme
    elif scelta == "2":
        return (50, 10)
    elif scelta == "3":
        return (100, 15)
```

Unpacking — spaccettare la tupla

```
# Assegnare i due valori a due variabili separate
limite, max_tentativi = scegli_difficolta()

print(limite)           # 20
print(max_tentativi)    # 8
```

L'unpacking e' il modo piu' naturale di usare le tuple restituite da una funzione: ogni valore va nella variabile corrispondente per posizione.

Accesso per indice

```
risultato = scegli_difficolta()
risultato[0]    # 20  (primo elemento)
risultato[1]    # 8   (secondo elemento)
```

3. Il ciclo **while** con contatore

while condizione: — ripete finche' la condizione e' vera

```
contatore = 0

while contatore < 5:
    print(f"Iterazione {contatore}")
    contatore += 1    # FONDAMENTALE: senza questo, il ciclo non finisce mai!

# Stampa: Iterazione 0, 1, 2, 3, 4
```

Nel progetto: limitare i tentativi

```
tentativi_usati = 0

while tentativi_usati < max_tentativi:
    # Chiedi un tentativo
    tentativo = chiedi_tentativo(1, limite)

    if tentativo is None:
        continue    # Input non valido, non conta

    tentativi_usati += 1    # Incrementa solo per tentativi validi

    # Verifica se ha indovinato
    if tentativo == numero_segreto:
        print(f"Hai indovinato in {tentativi_usati} tentativi!")
        break    # Esci dal ciclo
```

while True con break

Un pattern alternativo per cicli che terminano in base a una condizione interna:

```
while True:
    risposta = input("Vuoi continuare? (s/n): ")
    if risposta == "n":
        break    # Esce dal ciclo
```

Differenza: `while condizione` vs `while True`

Pattern	Quando usarlo
<code>while contatore < max:</code>	Sai in anticipo quante iterazioni
<code>while True: + break</code>	Non sai quando l'utente vorrà fermarsi

4. `continue` — saltare un'iterazione

`continue` salta il resto del corpo del ciclo e torna all'inizio:

```
while tentativi_usati < max_tentativi:
    tentativo = chiedi_tentativo(1, limite)

    if tentativo is None:
        continue    # ← Salta TUTTO il codice sotto e torna al while
                    #   Il contatore NON viene incrementato!

    tentativi_usati += 1    # Questo viene eseguito solo se tentativo e' valido
    # ...
```

`break` vs `continue`

Istruzione	Effetto
<code>break</code>	Esce dal ciclo completamente
<code>continue</code>	Salta al prossimo giro del ciclo

```
for i in range(5):
    if i == 2:
```

```
        continue      # Salta il 2, ma continua con 3, 4
    if i == 4:
        break          # Esce dal ciclo al 4
    print(i)
# Stampa: 0, 1, 3
```

5. Condizionali: confronto numerico

Operatori di confronto

Operatore	Significato
<code>==</code>	Uguale a
<code>!=</code>	Diverso da
<code>></code>	Maggiore di
<code><</code>	Minore di
<code>>=</code>	Maggiore o uguale
<code><=</code>	Minore o uguale

Nel progetto: dare suggerimenti

```
def dai_suggerimento(tentativo, numero_segreto):
    if tentativo > numero_segreto:
        print("Troppo alto! Prova piu' basso.")
        return False
    elif tentativo < numero_segreto:
        print("Troppo basso! Prova piu' alto.")
        return False
    else:
        # tentativo == numero_segreto
        return True
```

Validare un range

```
# Verificare che un numero sia nel range [1, 20]
if valore < 1 or valore > 20:
    print("Fuori range!")

# Forma equivalente e piu' leggibile
if not (1 <= valore <= 20):
    print("Fuori range!")
```

6. Conversione di tipo: `int()` e `try/except`

`int()` — da stringa a intero

```
int("42")      # 42
int("7")       # 7
int("abc")     # ValueError! Non e' un numero
int("3.14")    # ValueError! int() non accetta decimali
```

`try/except` — gestire gli errori

```
try:
    valore = int(input("Inserisci un numero: "))
    print(f"Hai inserito: {valore}")
except ValueError:
    print("Non e' un numero valido!")
```

Come funziona:

1. Python **prova** a eseguire il codice nel blocco `try`
2. Se `int()` solleva un `ValueError`, il programma salta al blocco `except`
3. Se non c'e' errore, il blocco `except` viene ignorato

Nel progetto


```
def chiedi_tentativo(minimo, massimo):
    try:
        valore = int(input(f"Inserisci un numero ({minimo}-{massimo}): "))
        if valore < minimo or valore > massimo:
            print(f"Il numero deve essere tra {minimo} e {massimo}.")
            return None
        return valore
    except ValueError:
        print("Devi inserire un numero intero.")
        return None
```

Restituire `None` segnala "input non valido" al codice chiamante, che può usare `if tentativo is None: continue`.

Riferimento: [try/except](#)

7. Liste e `.append()` per raccogliere risultati

Liste — ripasso

```
risultati = []           # Lista vuota
risultati.append(3)      # [3]
risultati.append(5)      # [3, 5]
risultati.append(-1)     # [3, 5, -1]
risultati.append(2)      # [3, 5, -1, 2]

len(risultati)           # 4
```

Nel progetto: raccogliere i risultati delle partite

```
risultati = []           # Lista che raccoglie il risultato di ogni partita

# Dopo ogni partita:
risultato = gioca_partita()  # es: 3 (vinto in 3 tentativi) o -1 (perso)
risultati.append(risultato)
```

```
# Dopo tutte le partite:  
mostra_statistiche(risultati)    # Analizza la lista completa
```

La convenzione del progetto:

- **Valore positivo** (es: 3, 5, 2) = vittoria, il numero indica i tentativi usati
- **Valore -1** = sconfitta

8. List comprehension per filtrare

Filtrare gli elementi di una lista

```
risultati = [3, 5, -1, 2, -1, 4]  
  
# Solo i valori positivi (vittorie)  
vittorie = [r for r in risultati if r > 0]  
# [3, 5, 2, 4]  
  
# Solo le sconfitte  
sconfitte = [r for r in risultati if r == -1]  
# [-1, -1]
```

Come si legge

```
[r for r in risultati if r > 0]  
# "prendi ogni r dalla lista risultati, ma solo se r > 0"
```

9. `sum()`, `len()`, `min()` — statistiche base

`sum()` — somma di tutti gli elementi

```
vittorie = [3, 5, 2, 4]
sum(vittorie)      # 14
```

len() — numero di elementi

```
len(vittorie)      # 4
```

Media: sum() / len()

```
media = sum(vittorie) / len(vittorie)      # 14 / 4 = 3.5
```

Attenzione: divisione per zero! Se la lista e' vuota, len() restituisce 0.

```
# SBAGLIATO (se vittorie e' vuota):
media = sum(vittorie) / len(vittorie)      # ZeroDivisionError!

# CORRETTO:
if vittorie:      # lista non vuota = True
    media = sum(vittorie) / len(vittorie)
```

min() — valore minimo

```
min(vittorie)      # 2 (il miglior risultato: meno tentativi)
```

Formattazione con :.1f

```
media = 3.5
print(f"Media: {media:.1f}")      # "Media: 3.5" (1 decimale)
print(f"Media: {media:.2f}")      # "Media: 3.50" (2 decimali)
```

10. Metodi delle stringhe: `.strip()` e `.lower()`

`.strip()` — rimuovere spazi

```
" ciao ".strip()      # "ciao"
" s ".strip()         # "s"
"n\n".strip()         # "n"
```

`.lower()` — tutto minuscolo

```
"S".lower()          # "s"
"Si".lower()          # "si"
"N".lower()           # "n"
```

Combinazione per normalizzare l'input

```
risposta = input("Vuoi giocare ancora? (s/n): ").strip().lower()
# L'utente puo' digitare "S", " s ", "Si", " SI " → diventa tutto "s" o "si"

if risposta != "s" and risposta != "si":
    break    # Esce solo se NON ha detto si
```

Catena di metodi

`.strip().lower()` applica prima `strip()` e poi `lower()` al risultato. Funziona perché ogni metodo restituisce una nuova stringa su cui puoi chiamare il metodo successivo.

11. Riepilogo e consigli

Mappa concetti → dove li usi nel progetto

Concetto	Dove lo usi
<code>random.randint(1, limite)</code>	<code>genera_numero()</code> — crea il numero segreto
Tupla come return	<code>scegli_difficolta()</code> → (<code>limite</code> , <code>max_tentativi</code>)
Unpacking	<code>limite, max_tentativi = scegli_difficolta()</code>
<code>while contatore < max:</code>	Ciclo dei tentativi in <code>gioca_partita()</code>
<code>while True: + break</code>	Ciclo delle partite in <code>main()</code>
<code>continue</code>	Saltare i tentativi con input non valido
<code>if/elif/else</code>	Suggerimenti in <code>dai_suggerimento()</code>
<code>int() + try/except</code>	Validazione input in <code>chiedi_tentativo()</code>
<code>return None</code>	Segnalare input non valido
<code>.append()</code>	Raccogliere risultati delle partite
List comprehension + filtro	Filtrare vittorie e sconfitte
<code>sum()/len()</code>	Calcolare la media
<code>min()</code>	Trovare il miglior risultato
<code>.strip().lower()</code>	Normalizzare la risposta s/n

Ordine consigliato di implementazione

1. `genera_numero(limite)` — una riga, la base del gioco
2. `scegli_difficolta()` — menu con if/elif/else, restituisce tupla
3. `chiedi_tentativo(min, max)` — input con validazione
4. `dai_suggerimento(tentativo, segreto)` — confronto con if/elif/else
5. `gioca_partita()` — mette insieme 1-4 con il ciclo while
6. `mostra_statistiche(risultati)` — analisi della lista
7. `main()` — ciclo esterno delle partite + statistiche finali

Errori comuni

- **Dimenticare `import random`** — senza import, `random.randint` non funziona
 - **Ciclo infinito** — dimenticare di incrementare il contatore o di usare `break`
 - **Divisione per zero** — calcolare la media su una lista vuota
 - `int("3.14")` — `int()` non accetta stringhe con punto decimale, usa `float()` se necessario
 - **Contare tentativi non validi** — un `continue` prima dell'incremento li esclude
-

Riferimenti ufficiali

- `random.randint()`
- [Funzioni](#)
- `while`
- `break` e `continue`
- `try/except`
- [Liste](#)
- [f-string](#)